Guía del alumno

Módulo 5, Aprendizaje no supervisado

Ruta guiada

Análisis Avanzado de Datos con Python

Cómo usar esta guía

Esta guía acompaña al Lab 05 guiado y sigue sus dos secciones en el mismo orden y con los mismos nombres del cuaderno. La sección En clase tiene tres partes, de Parte 1 de 3 a Parte 3 de 3, y se hace en clase. La sección Para estudiar tiene tres estudios, de Estudio 1 de 3 a Estudio 3 de 3, y se hace después de clase con esta guía, una vez corrida la sección En clase. Cada parte trae el código y la respuesta tal como aparecen en Colab, y una explicación en palabras simples.

En las partes no hay que escribir código. Lo que cambia se cambia en las listas desplegables y después se vuelve a correr la celda. El lab responde tres preguntas. Si solo miramos cómo generan durante el día, qué centrales se parecen entre sí. Cómo se juntan en grupos sin que nadie diga de antemano cuáles son. Y qué centrales no se parecen a nada, y por qué eso también es un hallazgo.

Lo que vas a poder hacer al terminar

- Describir una central con su día promedio, 24 números, y compararla por la forma y no por el tamaño.
- Pedirle a K-Means una cantidad de grupos y leer qué juntó.
- Decidir cuántos grupos pedir mirando hasta dónde baja la inercia.
- Usar DBSCAN cuando no se sabe cuántos grupos hay, y leer su ruido como un hallazgo.
- Resumir muchos números en dos con PCA, medir los grupos con la silueta y encontrar temas en texto con LDA.

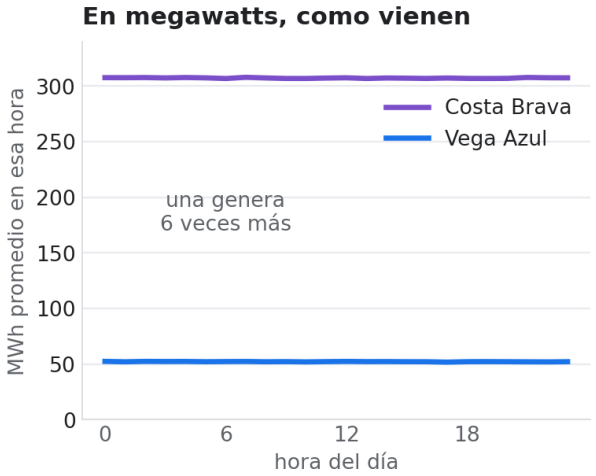
Todas las respuestas de esta guía vienen de una corrida del cuaderno con los valores que trae por defecto. Los datos del curso son sintéticos, y las tecnologías se separan tan limpio porque la generación de cada central se creó a partir de un día tipo por tecnología.

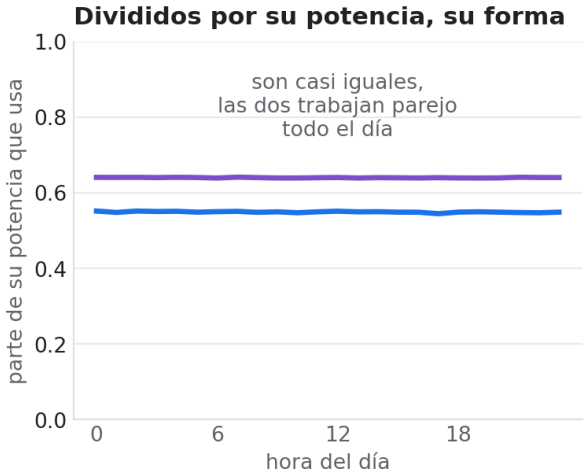
En clase

Tres partes, una detrás de la otra, que se hacen en clase en el mismo orden del cuaderno.

Parte 1 de 3 · Comparar por la forma, no por el tamaño

Una central grande y una chica pueden trabajar igual, parejo todo el día, y aun así verse distintas si se las compara en megawatts. Para comparar cómo trabajan, y no cuánto, cada central se describe con su día promedio, el promedio de cada hora durante 2024, y se divide por su potencia. Así cada número dice qué parte de su potencia usa a esa hora, y todas quedan entre 0 y 1.





Parte 1 de 3

```
# Parte 1
DATOS = "https://raw.githubusercontent.com/RodrigoMSB/curso-python-minergia/main/datos/"
import pandas as pd

# La generación de cada hora de 2024 y la ficha de las 20 centrales, como en el Lab 02.
gen = pd.read_csv(DATOS + "generacion.csv", parse_dates=["fecha"])
centrales = pd.read_csv(DATOS + "centrales.csv")

# Una fila por central con 24 columnas, el promedio de cada hora. Es su día promedio.
perfil = gen.pivot_table(index="central", columns="hora", values="mwh", aggfunc="mean")

# Dividir cada fila por la potencia de esa central. Ahora cada número es la parte
# de su potencia que usa en esa hora, entre 0 y 1. Se compara la forma, no el tamaño.
potencia = centrales.set_index("central")["potencia_mw"]
factor = perfil.div(potencia, axis=0)

print("Centrales y horas", factor.shape)
print("Antes, de", round(perfil.values.min(), 1), "a", round(perfil.values.max(), 1), "MWh")
print("Después, de", round(factor.values.min(), 3), "a", round(factor.values.max(), 3))

# Una central de cada tecnología, de las 9 a las 14 horas.
tecnologia = centrales.set_index("central")["tecnologia"]
muestra = factor.index.to_series().groupby(tecnologia[factor.index]).first()
factor.loc[muestra].round(2).iloc[:, 9:15]
```

```
Centrales y horas (20, 24)
Antes, de 0.0 a 307.9 MWh
Después, de 0.0 a 0.666
hora                          9     10    11    12    13    14
central
Carboeléctrica Costa Brava  0.64  0.64  0.64  0.64  0.64  0.64
Diésel Respaldo Cordillera  0.00  0.00  0.00  0.00  0.00  0.00
Eólica Campo Abierto        0.31  0.31  0.32  0.31  0.32  0.30
Termoeléctrica Bahía Norte  0.67  0.66  0.66  0.66  0.66  0.66
Central Aguas Claras        0.55  0.55  0.55  0.55  0.55  0.55
Parque Solar Altiplano      0.16  0.21  0.25  0.26  0.26  0.23
```

Son 20 centrales y 24 horas. En megawatts, los números van de 0 a 307,9, y una central grande aplasta a las chicas. Divididos por la potencia van de 0 a 0,666. En la tabla se ve una central de cada tecnología. El carbón, el gas y la hidro están parejos de 9 a 14, la solar sube hacia el mediodía y el diésel está en cero porque solo entra en la tarde. Esa forma es lo que se va a comparar.

Parte 2 de 3 · K-Means, juntar lo que se parece

K-Means es un algoritmo que recibe una cantidad de grupos y reparte las filas entre ellos, de modo que las que quedan juntas se parezcan. Antes de dárselas, StandardScaler deja cada hora con promedio cero y desviación uno, para que ninguna hora pese más que otra. K-Means nunca ve la tecnología. La tabla la cruza con el grupo que eligió.



Parte 2 de 3

```
# Parte 2
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# X es la tabla factor de la Parte 1 de 3, pasada por StandardScaler. Eso deja cada hora
# centrada en cero y con el mismo ancho, para que ninguna hora pese más que otra.
X = StandardScaler().fit_transform(factor)

# K-Means pide cuántos grupos. Reparte las centrales y ajusta hasta que nada cambia.
km6 = KMeans(n_clusters=6, random_state=2026, n_init=10).fit(X)

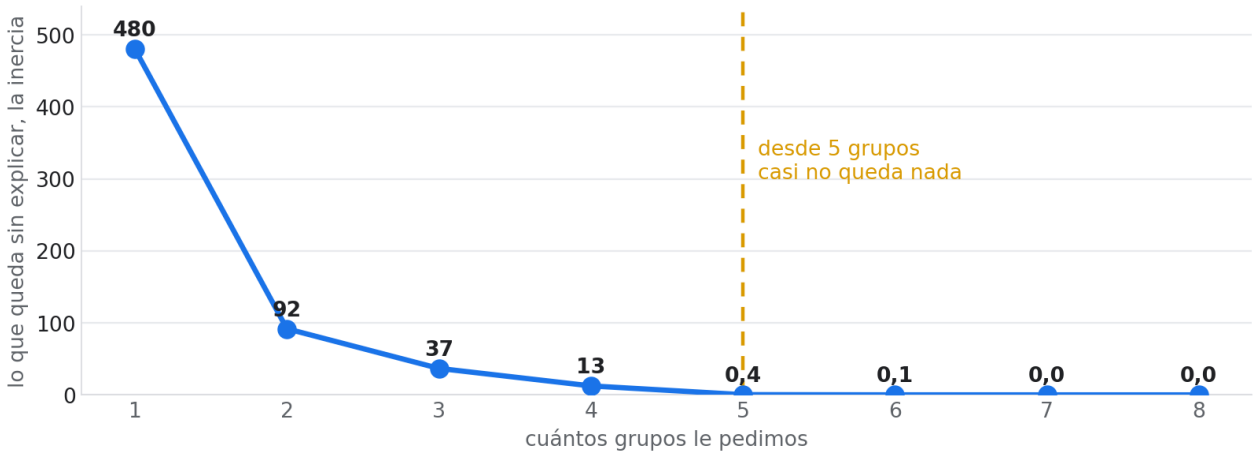
# La tecnología real, que K-Means no vio. Cada fila una tecnología, cada columna un grupo.
pd.crosstab(tecnologia[factor.index], km6.labels_)
```

```
col_0       0  1  2  3  4  5
tecnologia
carbon      0  0  0  0  0  2
diesel      0  0  0  1  0  0
eolica      0  0  4  0  0  0
gas         0  0  0  0  3  0
hidro       5  0  0  0  0  0
solar       0  5  0  0  0  0
```

Cada tecnología quedó en su propio grupo. Las 5 hidro en el grupo 0, las 5 solares en el 1, las 4 eólicas en el 2, el diésel en el 3, las 3 de gas en el 4 y las 2 de carbón en el 5. Nadie le dijo a K-Means qué era cada central. Solo con la forma del día las separó igual que lo haría una persona que conoce las centrales.

Tu turno · ¿Cuántos grupos pedir?

Cuántos grupos pedir es la decisión más importante de K-Means y nadie la sabe de antemano. Se prueba con varios y se mira cuánto desorden queda dentro de los grupos, lo que el cuaderno llama inercia. Siempre baja al agregar grupos. Lo que se busca es hasta dónde vale la pena. La lista desplegable permite probar con 4, 5, 6 o 7.



Parte 2 de 3, tu turno

```
# Parte 2, tu turno
grupos = 6 #@param [4, 5, 6, 7]

# Para cada cantidad de grupos, cuánto desorden queda dentro de los montones.
# El programa le llama inercia. Siempre baja al agregar grupos.
for k in range(1, 9):
    inercia = KMeans(n_clusters=k, random_state=2026, n_init=10).fit(X).inertia_
    print(k, "grupos, inercia", round(inercia, 1))

# Y la tabla con la cantidad de grupos elegida.
km = KMeans(n_clusters=int(grupos), random_state=2026, n_init=10).fit(X)
print()
print("Con", grupos, "grupos")
pd.crosstab(tecnologia[factor.index], km.labels_)
```

```
1 grupos, inercia 480.0
2 grupos, inercia 91.8
3 grupos, inercia 36.8
4 grupos, inercia 12.5
5 grupos, inercia 0.4
6 grupos, inercia 0.1
7 grupos, inercia 0.0
8 grupos, inercia 0.0

Con 6 grupos
col_0       0  1  2  3  4  5
tecnologia
carbon      0  0  0  0  0  2
diesel      0  0  0  1  0  0
eolica      0  0  4  0  0  0
gas         0  0  0  0  3  0
hidro       5  0  0  0  0  0
solar       0  5  0  0  0  0
```

La inercia cae de 480 a 92 con 2 grupos y a 37 con 3. Desde 5 ya casi es cero. Con 6 grupos la tabla es la misma de la parte anterior. Si se cambia a 4, el carbón, el gas y la hidro quedan en un solo grupo, porque las tres trabajan parejo todo el día, y solar, eólica y diésel quedan cada una en el suyo. Las dos respuestas son razonables. Con 4 se agrupa por cómo trabaja, con 6 se agrupa por tecnología.

> **CUÁNTOS GRUPOS PEDIR ES UNA DECISIÓN**
>
> Con 4 grupos, carbón, gas e hidro quedan juntas, porque las tres trabajan parejo todo el día. Con 6 grupos cada tecnología queda en su propio grupo, sin que nadie se lo dijera. La inercia cae fuerte hasta 3 grupos y desde 5 ya casi es cero. Se busca hasta dónde vale la pena agregar grupos. Si se agrupara en megawatts sin dividir por la potencia, K-Means juntaría grandes con grandes y las tecnologías saldrían mezcladas.

Parte 3 de 3 · DBSCAN, y lo que no cabe en ningún grupo

Hasta aquí cada central era una fila con su día promedio. Ahora cada central tiene una fila por día, 20 centrales por 366 días, 7.320 filas. DBSCAN es otro algoritmo de grupos que no pide cuántos. Pide un radio, eps, qué tan cerca tienen que estar dos filas para ser vecinas, y cuántos vecinos hacen falta para formar un grupo, min_samples. Las filas sin suficientes vecinos quedan con la etiqueta menos uno, el ruido.

tecnología	días en la tabla	en un grupo	fuera de todo grupo, ruido
hidro	1830	1830	0
gas	1098	1098	0
carbón	732	732	0
solar	1830	1830	0
diésel	366	366	0
eólica	1464	0	1464

1.464 es exactamente 4 centrales eólicas por 366 días. Ningún día de una eólica se parece lo suficiente a otro como para formar un grupo.

Parte 3 de 3

```
# Parte 3
from sklearn.cluster import DBSCAN

# Ahora una fila por central y día. 20 centrales por 366 días son 7.320 filas.
perfil_dia = gen.pivot_table(index=["central", "fecha"], columns="hora", values="mwh",
↪  aggfunc="mean")
potencia_dia = perfil_dia.index.get_level_values("central").map(potencia)
factor_dia = perfil_dia.div(potencia_dia, axis=0)
X_dia = StandardScaler().fit_transform(factor_dia)
print("Filas y horas", X_dia.shape)

# DBSCAN no pide grupos. Pide un radio, eps, qué tan cerca tienen que estar dos días
# para ser vecinos, y cuántos vecinos hacen falta para formar un grupo, min_samples.
# Los días sin suficientes vecinos quedan con la etiqueta menos uno, el ruido.
db = DBSCAN(eps=1.5, min_samples=5).fit(X_dia)

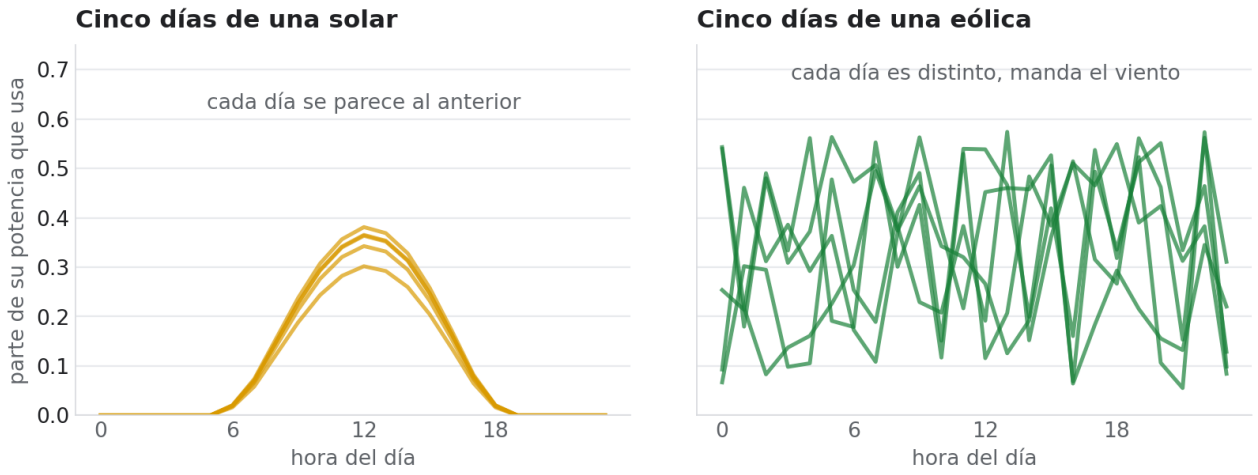
tecnologia_dia = perfil_dia.index.get_level_values("central").map(tecnologia)
pd.crosstab(tecnologia_dia, db.labels_)
```

```
Filas y horas (7320, 24)
col_0     -1     0    1     2
row_0
carbon     0   732    0     0
diesel     0     0  366     0
eolica  1464     0    0     0
gas        0  1098    0     0
hidro      0  1830    0     0
solar      0     0    0  1830
```

Con radio 1,5 DBSCAN armó 3 grupos. El carbón, el gas y la hidro quedaron juntos en el grupo 0, el diésel en el 1 y la solar en el 2. Y los 1.464 días de las eólicas quedaron fuera de todo grupo. Es exactamente 4 centrales por 366 días. Ningún día de una eólica se parece lo suficiente a otro, porque su producción la manda el viento.

Tu turno · Mueve el radio

El radio es la decisión de DBSCAN, como la cantidad de grupos lo era de K-Means. La lista permite probar 1,5, 2,0, 3,0 y 3,5. Un radio chico deja más filas fuera, un radio grande mete a todos en algún grupo.



Parte 3 de 3, tu turno

```
# Parte 3, tu turno
radio = 1.5 #@param [1.5, 2.0, 3.0, 3.5]

prueba = DBSCAN(eps=float(radio), min_samples=5).fit(X_dia)
cuantos = len(set(prueba.labels_)) - (1 if -1 in prueba.labels_ else 0)
ruido = int((prueba.labels_ == -1).sum())

print("Radio", radio)
print("Grupos", cuantos)
print("Días fuera de todo grupo, ruido", ruido, "de", len(X_dia))
print("Días eólicos en la tabla", int((tecnologia_dia == "eolica").sum()))
```

```
Radio 1.5
Grupos 3
Días fuera de todo grupo, ruido 1464 de 7320
Días eólicos en la tabla 1464
```

Con 1,5 quedan 1.464 días de ruido, los de las eólicas. Con 3,0 quedan 5 y con 3,5 ya no queda ninguno, pero no porque las eólicas se hayan vuelto parejas, sino porque el radio es tan grande que acepta cualquier vecino. El ruido se mira antes de descartarlo. Aquí es la respuesta a la tercera pregunta del lab.

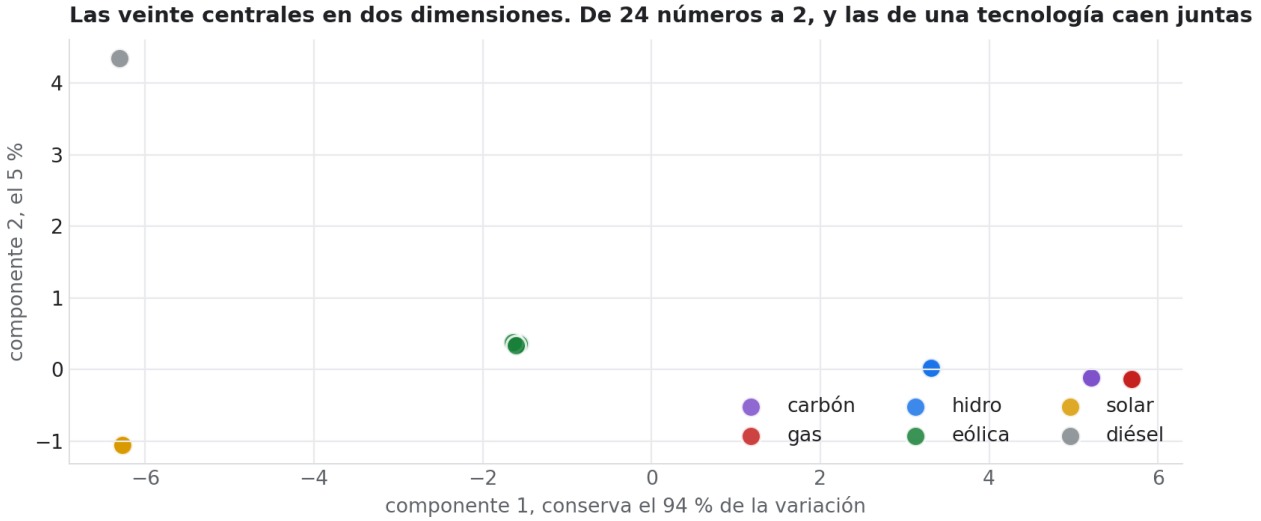
> **EL RUIDO NO SIEMPRE ES UN ERROR**
>
> Con radio 1,5 quedan 1.464 días fuera de todo grupo. Son exactamente las 4 eólicas por 366 días. Ningún día de una eólica se parece lo suficiente a otro como para formar un grupo, porque su producción la manda el viento. Con radio 3,5 ya no queda ruido, pero porque el radio es tan grande que mete a todos en algún grupo y se pierde el hallazgo. El ruido se mira antes de descartarlo. Aquí es la respuesta a la tercera pregunta del lab.

Para estudiar

Estos estudios se hacen después de clase con esta guía. Antes de empezar, hay que correr las tres partes de En clase, porque usan sus tablas.

Estudio 1 de 3 · ¿Cómo se dibujan 24 números en un plano?

Cada central tiene 24 números y en un papel caben dos. PCA busca las dos direcciones en que los datos más se estiran y se queda con esas, de modo que al dibujar se pierda lo menos posible.



Estudio 1 de 3

```
# Estudio 1
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# PCA busca las dos direcciones donde los datos más se estiran y se queda con esas.
pca = PCA(n_components=2).fit(X)
Z = pca.transform(X)
print("Cuánto de la variación conservan las dos componentes",
↪  pca.explained_variance_ratio_.round(3))

mapa = pd.DataFrame(Z, columns=["pc1", "pc2"], index=factor.index)
mapa["tecnologia"] = tecnologia[mapa.index].values

fig, ax = plt.subplots(figsize=(7, 5))
for t, g in mapa.groupby("tecnologia"):
    ax.scatter(g["pc1"], g["pc2"], label=t, s=45, alpha=0.75)
ax.set_title("Las veinte centrales en dos dimensiones")
ax.set_xlabel("componente 1")
ax.set_ylabel("componente 2")
ax.legend()
plt.show()
```

```
Cuánto de la variación conservan las dos componentes [0.943 0.052]
```

Las dos componentes conservan el 94,3 y el 5,2 por ciento de la variación, casi todo. En el gráfico las centrales de una misma tecnología caen casi en el mismo punto, y las que trabajan parejo todo el día, carbón, gas e hidro, quedan a la derecha. PCA no agrupa, solo dibuja. Pero cuando se dibuja bien, los grupos se ven a simple vista.

Estudio 2 de 3 · ¿Los grupos quedaron bien?

La silueta mide, para cada central, qué tan cerca está de su grupo comparado con el grupo vecino. Va de menos uno a uno, cerca de uno es un grupo bien separado, y no necesita la respuesta correcta.



Estudio 2 de 3

```
# Estudio 2
from sklearn.metrics import silhouette_score

# La silueta mide, para cada punto, qué tan cerca está de su grupo comparado con
# el grupo vecino. Cerca de 1 es un grupo bien separado. No necesita la respuesta.
for k in range(2, 8):
    etiquetas = KMeans(n_clusters=k, random_state=2026, n_init=10).fit_predict(X)
    print(k, "grupos, silueta", round(silhouette_score(X, etiquetas), 3))
```

```
2 grupos, silueta 0.72
3 grupos, silueta 0.783
4 grupos, silueta 0.83
5 grupos, silueta 0.899
6 grupos, silueta 0.917
7 grupos, silueta 0.757
```

La silueta sube de 0,72 con 2 grupos a 0,917 con 6, y con 7 vuelve a caer a 0,757. Dice lo mismo que la inercia de la Parte 2 de 3, que 6 grupos es donde conviene parar, pero con una ventaja. La inercia siempre baja, la silueta tiene un máximo.

Estudio 3 de 3 · Temas en el texto libre

La bitácora del Lab 04 trae una columna con el texto que escribe el turno. LDA es un algoritmo que busca temas en un montón de textos sin leerlos. Primero cada texto pasa a ser un conteo de palabras, sacando las que están en todas partes, y después LDA reparte las palabras en la cantidad de temas que se le pide. La lista permite probar con 3 o 6.

tipo de evento	tema 0	tema 1	tema 2	tema 3	tema 4	tema 5
correctivo			**118**			
evento climático	**80**					
falla eléctrica					47	
falla mecánica		40				58
inspección				25	8	**66**
preventivo		**73**		**85**		

Cada fila es un tipo de evento de la bitácora. Cada columna, un tema que LDA armó solo con las palabras. Casi todos los tipos caen en uno o dos temas.

Estudio 3 de 3

```
# Estudio 3
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

temas = 6 #@param [3, 6]

# La bitácora del Lab 04, con la columna de texto que no habíamos usado.
mant = pd.read_csv(DATOS + "mantenimiento.csv")

# Cada observación pasa a ser un conteo de palabras. Las palabras que están en todas
# partes no distinguen nada, y se sacan.
VACIAS = ["el", "la", "los", "las", "un", "una", "de", "del", "en", "y", "se", "que", "por", "con",
          "sobre", "tras", "su", "sus", "al", "lo", "es", "durante", "ronda", "turno", "registra",
          "detecta", "reporta", "evento", "operador", "revisa"]
# max_df saca las palabras que están en más del 90 % de los textos, min_df las que aparecen en
↪  menos de 3.
vectorizador = CountVectorizer(stop_words=VACIAS, max_df=0.9, min_df=3)
matriz = vectorizador.fit_transform(mant["observacion"])

# LDA busca temas. No sabe que existen seis tipos de evento.
lda = LatentDirichletAllocation(n_components=int(temas), random_state=2026).fit(matriz)
palabras = vectorizador.get_feature_names_out()
for i, tema in enumerate(lda.components_):
    print(f"tema {i}:", ", ".join(palabras[j] for j in tema.argsort()[-6:][::-1]))

# A cada texto su tema dominante, cruzado con el tipo de evento, que LDA nunca vio.
dominante = lda.transform(matriz).argmax(axis=1)
pd.crosstab(mant["tipo_evento"], dominante)
```

```
tema 0: estructura, reaprietan, sella, patio, nieve, acumulación
tema 1: vibración, completa, pauta, observaciones, acoplamiento, sin
tema 2: servicio, orden, trabajo, pendiente, devuelve, repara
tema 3: filtros, rutina, lectura, ajuste, sin, cambio
tema 4: tensión, interruptor, protección, relé, elevada, corriente
tema 5: aceite, eje, rodamiento, programa, detalle, revisión
col_0              0   1    2   3   4   5
tipo_evento
correctivo         0   0  118   0   0   0
evento_climatico  80   0    0   0   0   0
falla_electrica    0   0    0   0  47   0
falla_mecanica     0  40    0   0   0  58
inspeccion         0   0    0  25   8  66
preventivo         0  73    0  85   0   0
```

Con 6 temas, cada tipo de evento cae en uno o dos. Los 118 correctivos en el tema 2, los 80 eventos climáticos en el 0, las 47 fallas eléctricas en el 4. Las fallas mecánicas se reparten entre el 1 y el 5, las inspecciones entre el 3 y el 5 con 8 en el 4, y los preventivos entre el 1 y el 3. LDA nunca vio el tipo de evento. Lo reconstruyó solo con las palabras, y las seis palabras de cada tema dicen por qué.

AGRUPAR SIN RESPUESTA CORRECTA

K-Means, DBSCAN y LDA encontraron grupos sin que nadie les dijera cuáles eran. En el curso podemos comparar contra la tecnología o el tipo de evento, porque los datos son inventados y los conocemos. En el trabajo real esa comparación no existe. Por eso se usan el codo, la silueta y el criterio de quien conoce el tema.

Lo que lograste hoy

- Cada central se puede describir con 24 números, su día promedio, y se compara por la forma dividiendo por la potencia.
- K-Means junta lo que se parece. Tú le dices cuántos grupos, él reparte hasta que queda estable.
- Con 6 grupos y comparando por la forma, recuperó las 6 tecnologías sin conocerlas.
- Cuántos grupos pedir se decide mirando hasta dónde la inercia deja de bajar.
- DBSCAN no pide grupos, pide un radio, y lo que no cabe en ninguno lo llama ruido.
- El ruido se mira antes de descartarlo. Aquí fueron los 1.464 días de las eólicas.

Para seguir practicando

Estos ejercicios son para quien quiera seguir después de clase. Aquí sí se escribe código. Antes de empezar, corre todo el cuaderno con Entorno de ejecución, Ejecutar todo. Escribe tu respuesta en la celda vacía de cada ejercicio y abre la solución solo después de intentarlo.

Ejercicio 1 · Los centros de cada grupo

El centro de un grupo es el promedio de sus centrales. Calcula el promedio de factor por grupo de km6 y mira las horas 0, 3, 12 y 21. Di cuál es el grupo de las solares y cuál el del diésel. Pista, factor.groupby(km6.labels_).mean() da una fila por grupo.

Solución · Ejercicio 1

```
centros = factor.groupby(km6.labels_).mean()
centros[[0, 3, 12, 21]].round(2)
```

El grupo de las solares es el único con cero a las 0, 3 y 21 horas y un valor alto a las 12. El del diésel es el único con cero a las 0, 3 y 12 y un valor alto a las 21. Los otros cuatro son parejos a toda hora, cada uno en su nivel, el de las eólicas más bajo.

Ejercicio 2 · K-Means sin dividir por la potencia

Repite la Parte 2 de 3 usando perfil en vez de factor, es decir, en megawatts tal como vienen. Cruza los 6 grupos con la tecnología y cuenta cuántos grupos mezclan tecnologías distintas.

Solución · Ejercicio 2

```
X_crudo = StandardScaler().fit_transform(perfil)
km_crudo = KMeans(n_clusters=6, random_state=2026, n_init=10).fit(X_crudo)
cruce = pd.crosstab(tecnologia[perfil.index], km_crudo.labels_)
print(cruce)
print("Grupos que mezclan tecnologías", int((cruce.gt(0).sum() > 1).sum()))
```

Las 5 hidro quedan repartidas en varios grupos y hay grupos con hidro, gas y carbón mezclados. Sin dividir por la potencia, K-Means agrupa por tamaño.

Ejercicio 3 · La silueta de DBSCAN

Calcula la silueta de la solución de DBSCAN con radio 3,0, sacando antes los puntos de ruido, y compárala con la de K-Means con 6 grupos en X. Pista, la silueta no acepta la etiqueta menos uno.

Solución · Ejercicio 3

```
db3 = DBSCAN(eps=3.0, min_samples=5).fit(X_dia)
sin_ruido = db3.labels_ != -1
print("Puntos que quedan", int(sin_ruido.sum()), "de", len(X_dia))
print("Silueta DBSCAN", round(silhouette_score(X_dia[sin_ruido], db3.labels_[sin_ruido]), 3))
print("Silueta K-Means 6", round(silhouette_score(X, km6.labels_), 3))
```

Son dos tablas distintas, una fila por central contra una fila por central y día, así que las siluetas no se comparan directo. Lo que importa es que DBSCAN deja 5 puntos fuera con radio 3,0 y K-Means no deja ninguno.

Para repasar

Referencia rápida

Lo que se escribe	Para qué sirve
`pivot_table(...)`	El día promedio, una fila por central y una columna por hora
`div(potencia, axis=0)`	Divide cada fila por la potencia de su central
`StandardScaler()`	Deja cada columna con promedio cero y desviación uno
`KMeans(n_clusters=6, ...)`	Reparte las filas en la cantidad de grupos que se le pide
`inertia_`	Cuánto desorden queda dentro de los grupos
`pd.crosstab(a, b)`	Cuenta las filas de cada combinación, por ejemplo tecnología y grupo
`DBSCAN(eps=1.5, min_samples=5)`	Grupos por vecindad, y la etiqueta menos uno para el ruido
`PCA(n_components=2)`	Resume muchas columnas en dos para poder dibujarlas
`silhouette_score(X, etiquetas)`	Qué tan separados quedaron los grupos, de menos uno a uno
`CountVectorizer` `LatentDirichletAllocation`	Cuenta las palabras de cada texto y busca temas

Si algo sale mal

Lo que aparece en pantalla	Qué pasó
`NameError: name 'factor' is not defined`	Corriste una celda sin correr antes la Parte 1 de 3, que es la que arma la tabla `factor`.
`NameError: name 'X' is not defined`	El tu turno de la Parte 2 y los estudios 1 y 2 usan `X`, que arma la Parte 2 de 3. Corre la Parte 2 antes.
`NameError: name 'X_dia' is not defined`	El tu turno de la Parte 3 usa la tabla de una fila por día que arma la Parte 3 de 3. Corre la Parte 3 antes.
La respuesta no cambió después de cambiar la lista	Cambiar un campo no corre la celda. Hay que volver a apretar el botón de play.

Tres preguntas de repaso

1. Por qué se divide el día promedio por la potencia antes de agrupar

Para comparar la forma y no el tamaño, si no K-Means junta grandes con grandes.

2. Qué diferencia hay entre lo que pide K-Means y lo que pide DBSCAN

K-Means pide cuántos grupos, DBSCAN pide un radio y decide solo cuántos grupos hay, y deja fuera lo que no cabe.

3. Qué significa que los 1.464 días de las eólicas quedaran como ruido

Que ningún día de una eólica se parece lo suficiente a otro, porque su producción la manda el viento. Es un hallazgo, no un error.

El Lab 05 guiado en una plana

Para imprimir y tener al lado durante el curso

Las partes y los estudios

Dónde	Qué se hace	Lo que se escribe
Parte 1 de 3	El día promedio de cada central, dividido por su potencia	`pivot_table`, `div`
Parte 2 de 3	K-Means con 6 grupos, y la inercia para elegir cuántos	`StandardScaler`, `KMeans`, `crosstab`
Parte 3 de 3	DBSCAN sobre 7.320 días, y el ruido de las eólicas	`DBSCAN`, `eps`, `min_samples`
Estudio 1 de 3	PCA, de 24 números a 2	`PCA`
Estudio 2 de 3	La silueta, el máximo dice cuántos grupos	`silhouette_score`
Estudio 3 de 3	LDA, temas en el texto de la bitácora	`CountVectorizer`, `LatentDirichletAllocation`

Cómo se elige

Cuántos grupos pedir a K-Means	Donde la inercia deja de bajar, o donde la silueta es máxima
Qué radio darle a DBSCAN	El que deja fuera lo que de verdad no se parece, no el que mete a todos
Qué hacer con el ruido	Mirarlo antes de descartarlo. Puede ser el hallazgo